Should Smart Homes Be Afraid of Evil Maids?

Identifying Vulnerabilities in IoT Device Firmware

Austen Knapp[co] Emmanuel Wamuo[co] Minhajul Alam Rahat[co]
Santiago Torres-Arias[in] Gedare Bloom[co] Yanyan Zhuang[co]
[co] University of Colorado, Colorado Springs [in] Purdue University

***Abstract*—The Internet of Things (IoT) revolution has transformed everyday consumer objects into interconnected, intelligent devices. Due to historically weak security designs, these devices are susceptible to compromises with far-reaching consequences. This paper aims to evaluate the current state of IoT device firmware security and assess the effectiveness of existing methods in safeguarding sensitive data. We conducted a manual analysis of 16 IoT devices, revealing a range of firmware management techniques, each varying in their effectiveness against an evil maid attack scenario. Out of the 16 devices, only 2 showed evidence of encrypting data at rest, despite 10 having crypto-enabled hardware. Additionally, 9 out of 16 devices possessed secure boot-enabled hardware, but 4 of them did not properly utilize or implement secure boot. Consequently, 13 devices were identified as vulnerable to sensitive data compromise, and 11 were at risk of firmware modification attacks. To address these critical security gaps, this study proposes a method for analyzing data-at-rest encryption and secure boot status in IoT devices. In this paper we shed light on the prevailing security shortcomings and provide practical analysis techniques to foster improved IoT device security and user data protection.**

***Index Terms*—Firmware Analysis, Data Encryption, Secure Boot**

I. Introduction

The Internet of Things (IoT) encompasses a vast array of internet-connected devices, ranging from simple sensors to critical infrastructure control systems. Examples include home automation tools like smart lighting, medical devices like insulin pumps, and infrastructure monitoring and control such as smart grids. With an estimated 27 billion IoT device connections by 2025, data generated is skyrocketing, projected to reach 79.4 zettabytes by the same year [1], [2]. This influx of data holds sensitive user information, intellectual property, and potential security-related data. IoT devices' cyber-physical nature enables them to interact with the physical world, bringing inherent risk alongside their utility. Cases like Stuxnet [3], the Idaho National Laboratory generator experiment [4], and malicious botnet [5] and DDoS attacks [6] demonstrate the potential for real-world damage. Robust security measures for all connected devices are essential to preventing catastrophic consequences.

One of the challenges in IoT security is that an attacker capable of modifying the firmware can force a device to do anything within its physical capabilities. Device firmware is the collection of machine instructions, or code, executing on a device that governs its hardware and functionality. By nature, code executing at this level has unrestricted access to all connected peripherals and hardware. Arbitrary modification of the firmware is equivalent of having root access to the device.

In this paper, we systematically review (1) the encryption of data at rest and (2) the secure boot processes of IoT devices in an attempt to answer the question: *Should smart homes be scared of evil maids*? An evil maid attack [7] is an attack on an unattended device, where an attacker (an "evil maid") alters it in some undetectable way so that they can later access the device, or the data on it. Our work analyzes whether the above two protection mechanisms, or the lack thereof, can become a threat to the home and the users living within. By offering a comprehensive analysis, this work lays the foundation for improving the understanding of how to enhance firmware security, protect consumer data, and inform the wider research community.

We make the following contributions:

- We provide the first systematic evaluation of IoT device secure boot and encryption of data at rest.
- We provide procedures for analyzing secure boot and encryption of data at rest in IoT devices.
- We show the consequences of neglecting to use these protection mechanisms can be severe.

II. Background

In this section we discuss the foundational concepts of secure boot, encryption of data at rest, and the evil maid attack.

A. Evil Maid Attack

During an evil maid attack, an attacker has physical access to an unattended device. They wield the skill, tools, and time to stealthily extract data or subtly modify the device, evading immediate detection. This attack is plausible because many IoT devices are used in shared apartment buildings, hotels, schools, and offices. An attacker might gain physical access by posing as a technician or service provider, or even using insider privileges. They could sell used devices online with modified firmware or steal devices for data extraction.

Two prominent attacks possible in the evil maid threat model are (1) private data theft and (2) firmware modification. With physical access, an attacker can extract flash memory data, impacting device and user security based on the stored sensitive data. Similarly, firmware modification via physical

This work is supported in part by NSF grants OAC-2325369, OAC-2115134, Colorado State Bill 18-086, and the University of Colorado Faculty Revitalization Fellowship Program.

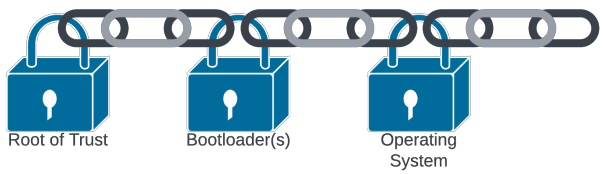


Fig. 1: Chain of Trust

access allows the attacker to gain complete control, potentially remote control upon successful boot-up of modified firmware. Defending against the two attacks requires implementing secure boot and encryption of data at rest.

B. Secure Boot

If a device implements secure boot, it establishes a chain of trust to ensure the integrity of executable images such as bootloaders and the kernel during startup. This chain forms as each stage verifies the next, anchoring the authenticity and integrity of each image to the trusted status of the preceding one. This chain of trust originates from a singular source of trusted information termed the "root of trust." In IoT, this foundational point can take various forms, including public keys embedded in the ROM (immutable memory), encryption keys stored in one-time-programmable (OTP) eFuses or Hardware Secure Modules (HSM), or use of a hardware component like a Trusted Platform Module (TPM) for cryptographic operations, or ARM's TrustZone. Figure 1 illustrates this chain of trust originating from a single, secure foundation. By implementing secure boot, device manufacturers ensure that only authorized and unaltered firmware can execute on their devices.

The absence or improper implementation of secure boot exposes IoT devices to severe vulnerabilities and potential breaches. Without secure boot, devices are susceptible to unauthorized or malicious code execution during startup, compromising their integrity and allowing attackers to gain control. This can lead to data breaches, unauthorized access, device manipulation, and even remote control by malicious actors. Moreover, the lack of proper secure boot implementation allows attackers to subvert the boot process, evade detection, and potentially exploit device vulnerabilities.

C. Encryption of Data at Rest

This mechanism adds an additional layer of security to sensitive information, preventing unauthorized access and data breaches. It is a crucial measure for safeguarding confidential data, including user information, proprietary data, and critical system files, against various threats such as theft, hacking, or unauthorized manipulation.

Key management is critical to effective encryption. If the encryption key is not stored or derived securely, it can potentially be recovered and used to obtain the plain text version of the storage data. Secure boot ensures that only trusted components, including the ones responsible for managing encryption keys, are executed. An attacker compromising the key management method would compromise the entire encryption process, highlighting the interdependence between secure boot and encryption key management. If secure boot is implemented incorrectly, the attacker could circumvent the storage encryption by getting the device to execute modified firmware. This could allow the attacker to retrieve the keys from the device or even employ the device to decrypt the storage. It is clear these two protection mechanisms work together in order to provide confidentiality and integrity.

III. Related Work

A. Secure Boot and Memory Encryption

To ensure the integrity of a secure system, it is necessary to implement secure boot. Previous works have proposed several secure boot techniques such as ARM-based secure boot using TrustZone technology [8], post-quantum digital signature-based secure boot [9], Trusted Platform Module-based secure boot [10], and secure reboot in real-time systems [11]. These techniques ensure a trusted execution environment of the runtime by preventing unauthorized code execution at startup.

Memory protection prevents attacks that aim to steal confidential data from memory, like the cold boot attack [12]. Recent work in memory protection frequently employ memory encryption techniques through three main approaches: hardware enhancements, operating system enhancements, and specialized industrial applications [13]. Cryptkeeper [14] reduces the amount of sensitive data in memory by allocating a smaller portion of RAM to store decrypted data for immediate use while the majority of RAM is encrypted. ZIA [15] implements encryption mechanism based on external beaconing token worn by the user, and encrypts the data in RAM depending on the user's vicinity. Freeze and Crypt [16] provides memory encryption based on the freezer functionality of the Linux kernel. HyperCrypt [17] employs a hypervisor-based encryption. CryptMe [18] combines memory encryption and ARM TrustZone-based memory access controls for memory protection. These memory encryption techniques are crucial to safeguard confidential data against physical attacks.

B. Firmware Analysis

Several works focus on firmware security analysis. Nadit et al. [19] summarized the state of firmware research in IoT. They provided insight into the many different firmware tools, analysis techniques, and firmware vulnerabilities. Neither development frameworks nor analysis tools can be standardized into one-size-fits-all solutions. Mtetwa et al. [20] reviewed firmware security threats, discussed the importance of firmware authenticity and integrity checks, and justified the need to verify firmware integrity after flashing it to storage. Bradley and Barrera [21] analyzed IoT network traffic and obtained a significant amount of firmware update related information (including some firmware packages) in real-time only from monitoring traffic. Gao et al. [22] examined a device that did not implement any encryption. The authors showed the importance of encrypting the firmware package in transit and implemented a secure update on a legacy MCU. Giese

and Noubir [23] showed how sensitive data can be obtained from used, factory-reset devices due to characteristics of flash wear-leveling and lack of encryption. Overall, scant work examines how device encryption and secure boot may impact the integrity of firmware. Though it is necessary that devices implement memory encryption and secure boot mechanisms to ensure security, there has not been much work in evaluating whether real world devices have these mechanisms in place.

IV. Methodology

In this section, we describe the methodology used in this study to systematically evaluate these protection mechanisms.

A. Device Storage Extraction

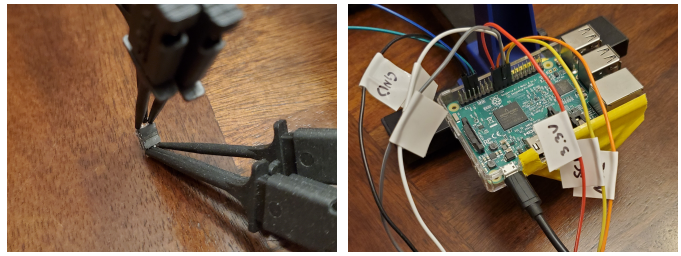
To study IoT device storage and boot procedure, we needed to obtain copies of the persistent storage. Different techniques are available for obtaining data from a particular device, and the techniques vary in difficulty. Our approach for data extraction was based on selecting the simplest available method. Generally speaking, storage data can be obtained through (1) root access, (2) bootloader memory dumps, (3) chip on-board programming, and (4) chip off-board memory dumps.

1) Root Access: Root access on a device can vary greatly in difficulty depending on the security of the device. In some cases, it simply requires connecting to the device over a serial connection such as UART if the root command prompt was left open. Manufacturers have also been known to use default, easily guessable, hard coded passwords [24]. In other cases, it can require circumventing startup scripts to get around device authentication. Once root access is gained, attackers gain the ability to effortlessly extract or manipulate data on the device.

2) Bootloader Memory Dump: Bootloader flash dumps can be performed without requiring root access to a device, if the manufacturer has not adequately secured the boot process. Many devices utilize secondary bootloaders, like U-Boot, to initiate the device before launching the kernel image. In some cases, gaining access to the U-Boot command prompt is feasible and U-Boot often offers tools for memory manipulation. When a bootloader is accessible on the device, it becomes possible to read and extract the flash data on a per-page basis. This process can be automated using a script to extract and reconstruct the flash content into a binary image of the chip.

3) Chip On-Board Programming: This method involves interacting directly with the storage chip while it remains soldered onto the device's circuit board. This can be achieved by using accessible ports on the board like JTAG or UART, though capabilities can differ based on manufacturer's design. Alternatively, it is possible to attach probes directly to exposed pins on the flash chip. To do this, the main System-on-Chip (SoC) or Microcontroller Unit (MCU) needs to be placed in a reset mode to prevent interaction with the flash chip. Then, a specialized programmer that communicates with the chip through methods like SPI can be employed to extract the data.

The on-board programming capability heavily relies on the specific device and its main SoC/MCU. Details about how to activate the reset mode or enter the flash mode are typically



(a) Chip Connections (b) Programming Connections

Fig. 2: Flash Storage Data Extraction

outlined in the chip's datasheet or on the manufacturer's website. For instance, Espressif devices can be booted into flash mode by grounding the GPIO0 pin [25]. In this mode, the device's flash memory can be read and written using tools like the manufacturer's open-source utility `esptool.py`. While this functionality and documentation may not be uniform across all chip manufacturers, some may provide similar methods for their devices.

4) Chip Off-Board Memory Dump: Chip off-board memory dumps entail physically removing the flash chip from the device's hardware to extract its data. This procedure involves de-soldering the chip from the board, establishing connections to the chip's leads or pads, and utilizing a chip programmer to read/write the data. For instance, Figure 2a illustrates probe connectors on a de-soldered flash chip. In some cases, direct wire soldering to the chip's pads is necessary. Figure 2b shows a Raspberry Pi, interfaced with the chip, setup for read/writes of the data using Flashrom with the SPI chip protocol. While dependable, this method is destructive, potentially preventing chip re-soldering onto the board. Throughout our analysis, this technique was often the most feasible method for retrieving persistent storage from devices.

B. Device Analysis

The investigation of each device adhered to the following standardized approach. First, cursory review of all devices was executed to identify device characteristics such as the SoC/MCU. Next, the device characteristics were used to aid storage extraction and provided information relevant to secure boot and encryption capabilities. We then analyzed each device's extracted storage to determine if any of them implemented storage encryption. Then, we attempted to determine if they were vulnerable to a firmware modification attack by an evil maid. If the device was determined to be vulnerable to a modified firmware attack, it did not use or properly implement secure boot.

1) Identifying Device Characteristics: Information needed for cursory exploration of the devices was the device architecture, operating system (OS), and chip information (SoC/MCU and flash/eMMC). We accomplished through physical means by opening the device to get access to the main board. We then used a microscope to identify markings on the chips, which identify the manufacturer and model of the chip. We could then find technical datasheets online for the particular chip. From

the datasheets, we could determine the device architecture and sometimes operating system. If the chip manufacturer provided a development framework, it was usually identified in the datasheet. This information was also found by searching the manufacturer's website for support material. In some cases, further exploration of the device storage data was required to identify the operating system.

2) Checking for Encryption of Data at Rest: For the purpose of evaluating if encryption was used, we considered a device to be unencrypted if hardware limitations prevent its use or if critical data was found in plain text. The methods we used were (1) hardware characterization, (2) file system analysis, and (3) manual inspection. All methods of analysis were applied in order to determine with certainty a device does not encrypt critical data. Each analysis method often required a different level of effort influenced by device characteristics such as storage chip type (eMMC or flash) and the OS. Linux based devices used a standard file system for storage on the flash chip. Espressif (Xtensa) devices used SPIFFS, a file system not found on other devices [26]. Other systems were less likely to contain a traditional file system. These systems required more manual analysis with tools such as `strings`, `hexdump`, and `Binwalk` entropy analysis to verify the flash was not encrypted.

Hardware characterization. First, we evaluated if the device was designed with encryption capabilities. This method is simple to apply by identifying the hardware components such as specialized crypto hardware on the device. With relevant datasheets, we identified if the hardware components were capable of implementing encryption. Although hardware characterization was useful, further analysis is still required to determine with certainty if encryption was implemented.

File system analysis. The next effort was to obtain executable code or known file system formats from the binary blob image of the device's persistent storage. Generally, the OS or development framework of the device greatly influenced the process for extracting relevant code. If we could extract file systems or mount partitions from flash image files, then critical data was very likely not encrypted. However, further manual inspection was required to identify critical data.

Manual inspection. If a file system was extracted, we searched through files and traced startup script execution flow behavior to identify sensitive data and files. This would lead us to scripts and executable files handling critical operations for the device such as firmware updates, logging, and communications. In some cases we identified keys being used to encrypt/decrypt firmware updates, sensitive user data such as WiFi passwords, private keys, and signatures being used for image verification.

If further inspection was necessary, we applied string searching and entropy analysis to both file names and content within files. String searching was completed using tools such as `hexdump` and `strings`. If we were able to determine readable information (such as file paths, usernames, passwords, network information, etc.) from the search, then that portion of the storage was determined not encrypted. We also used `Binwalk`'s entropy calculation to verify if all or certain portions of the data was encrypted. If the entropy calculation showed long sections of data with entropy close to one, then the portion of data was considered encrypted.

3) Checking for Secure Boot: We identified devices as not using, or not implementing correctly, secure boot if hardware limitations prevented it from being used or if the device would boot with modified firmware. The methods we used to determine this was (1) hardware characterization and (2) manual inspection.

Hardware characterization. We first checked if the device contained external secure hardware such as a TPM or HSM, as described in Section II-B. If nothing was found, the only remaining option was to evaluate the capabilities of the main SoC/MCU. Not all SoC/MCU options available are capable of building a chain of trust from a secure root. If we identified through manufacturer documentation (i.e., chip datasheets or developer documents) the chip was not capable of implementing secure boot, the device is clearly vulnerable to firmware modification attacks.

Manual inspection. Next, we performed manual analysis and inspection of any available scripts, code, or information to determine if any evidence supported that the device was using secure boot. If the device logged information over UART, we inspected the log to understand the boot process. We also inspected unencrypted files and partitions to identify information about potential implementations of secure boot. For example, if a device contained code which read the eFuse values, it meant the device manufacturer was very likely using this method as the root of trust. Also, common bootloaders such as U-Boot check for files with data to be available during the boot process in order to verify the integrity of the kernel. If these files were found, it was considered the device was using secure boot. This did not mean, however, the device implemented secure boot properly.

Finally, checking if secure boot was implemented properly could be done by attempting to boot the device with modified firmware. This means we would need to extract the device firmware, modify it, then write the modified firmware back to the device. If the device successfully booted after doing this, the device did not implement secure boot properly.

V. Security Analysis

Table I shows the characteristics of the 16 devices we analyzed. A wide range of chip (MCU/SoC) selection was used which directly influenced the architecture and often the OS. The most selected operating system was Linux, with the remaining devices using a real-time operating system (RTOS) or bare metal. The device architectures were mostly ARM with some being Espressif's Xtensa. Finally, a few devices were MIPS.

In Table I, many of the devices used Espressif's Xtensa. All of the flash data from devices with ESP chips was obtained using the open-source tool, `esptool.py`. This tool is used for communicating with the ROM bootloader of Espressif chips [27]. Almost all of the remaining device firmware was

TABLE I: Summary of device storage extraction method. Similary extraction methods are grouped together.

Device	Model	OS	Architecture	Storage Retrieval Method
Amazon Echo Show 5	1st Gen	Android	ARM	Chip Removal - eMMC Socket Reader
Amazon Echo Smart Speaker	SK705D1	Linux	ARM	
SmartThings Hub V2	STH-ETH-250	Linux	ARM	
SmartThings Hub V3	IM6001-V3P01	Linux	ARM	
Amcrest ProHD Shield	IPM-HX1B	Linux	ARM	Chip Removal - Flashrom
Honeywell Home	RTH9585WF	Bare Metal	ARM	
Ring Chime	5AT1S2	TI-RTOS	ARM	
Ring Bridge	5B01S8	Linux	MIPS	
Victure Camera	PC420	Linux	MIPS	
Govee Thermometer	WDM-1193	Bare Metal	Xtensa	UART esptool.py
iClever Air Quality Monitor	IC-CWA1W	Bare Metal	Xtensa	
Orbit bHyve	57925	FreeRTOS	Xtensa	
Gosund WiFi Smart Plug	WP6	Bare Metal	Xtensa	UART esptool.py, Tuya Convert
Wyzye Lock Gateway	WLGW01	FreeRTOS	Xtensa	UART esptool.py, Chip Removal - Flashrom
Philips Hue Bridge	3241312018A	Linux	MIPS	UART U-Boot NAND Dump
ReFoss	MSG100	FreeRTOS	ARM	Unsuccessful[†]

[†]No external flash, uses SoC on-chip flash storage.

TABLE II: Summary of device storage encryption. Model, OS, and architecture are the same as in Table I. ? means the results of the analysis were inconclusive.

Device Name	Crypto-Enabled SoC/MCU	Encrypted Storage
Amazon Echo Show 5	✓	×[a]
Amazon Echo Smart Speaker		×
Amcrest ProHD Shield		×
Orbit b-Hyve		×
ReFoss		?[b]
Ring Chime		×
SmartThings Hub V2		✓[c]
SmartThings Hub V3		✓
Victure Camera		×
Wyze Lock Gateway		×
Gosund WiFi Smart Plug	×	×
Govee Thermometer		
Honeywell Home		
iClever Air Quality Monitor		
Philips Hue Bridge		
Ring Bridge		

[a]Encryption status also verified by Giese and Noubir [23].
[b]The device only uses on-chip memory, making data extraction more difficult.
[c]One unencrypted partition contains potentially compromizing data.

obtained by physically removing the chip and utilizing a reader to dump the data. The only other exception was the Philips Hue Bridge, which we obtained over the U-Boot bootloader command prompt. With U-Boot's built in NAND dump utility, we extracted the flash chip image using a script over a UART serial connection. Finally, the ReFoss garage door opener did not have an external flash chip and only utilized the SoC on-chip flash storage. As a result, removing a flash chip to extract the storage data was not an option. We attempted to interact with the main SoC bootloader to extract the chip memory but were unsuccessful.

A. Case Studies

In this section, we show a few case studies during our security analysis. First, our review of the Ring Chime demonstrated potential for severe consequences due to a lack of security measures. We first implemented a HTTPS downgrade attack by modifying the device firmware to disable HTTPS, and booting the device using the modified firmware. This allowed us to identify and connect to Ring's update server using HTTP requests, and ultimately obtain over 500 firmware images across various device types in the Ring ecosystem. In another case, we identified the symmetric encryption key for firmware update packages in the Philips Hue Bridge's persistent storage. We verified the key by successfully decrypting a firmware package. The used Amazon Echo Show 5 we purchased contained network credentials of the previous owner. The Samsung SmartThings hubs used SoC OTP eFuses to establish a root of trust. Ultimately, this implementation prevented us from successfully decrypting the rootfs and user data partitions. Each device in our analysis showed varying levels of security practices and sensitive, or private, data residing in plain text.

B. Encryption of Data at Rest

Next, we provide aggregated results from our analysis. Table II shows the results from the analysis of encryption. Devices are grouped by whether or not they contained a crypto-enabled SoC/MCU. Devices with a ✓ in the "Crypto-Enabled SoC/MCU" column were verified to support cryptographic operations from the device datasheet or manufacturer's website. An × in the same column meant the opposite. A ✓ in the "Encrypted Storage" column represents a device which we verified to have some or all of the flash storage encrypted. An × meant we verified the device did not encrypt any significant portion of its flash storage.

Ten out of 16 devices had crypto-enabled SoC/MCU chips; however, only two of the devices implemented storage encryption. Each eMMC device (Amazon Echo, Echo Show 5, SmartThings Hubs) used standard Linux file system partition for its binary image. For the Amazon Echo smart speaker, we reviewed the device boot log and inspected the eMMC partitions. The boot log listed the names of each partition which provided their purpose. We then inspected the root file system and user data partitions and were able to mount and browse files for all of them concluding that the Amazon Echo

TABLE III: Summary of device secure boot. Model, OS, and architecture are the same as in Table I. ? means the results of the analysis were inconclusive.

Device Name	Secure Boot Enabled SoC/MCU	Secure Boot
Amazon Echo Show 5	✓	?[a]
Amazon Echo Smart Speaker		×
Amcrest ProHD Shield		×
Orbit b-Hyve		×
ReFoss		?[b]
Ring Bridge		?
SmartThings Hub V2		✓[c]
SmartThings Hub V3		✓[d]
Wyze Lock Gateway		×
Gosund WiFi Smart Plug	×	×
Govee Thermometer		
Honeywell Home		
iClever Air Quality Monitor		
Philips Hue Bridge		
Ring Chime		
Victure Camera		

[a]Not verified in this work, but verified by Giese and Noubir [23].
[b]The device only uses on-chip flash, making secure boot less critical.
[c,d]Evidence supports a U-Boot secure boot implementation.

smart speaker did not encrypt critical data at rest. Using similar method we determined that the Amazon Echo Show 5 did not implement storage encryption. The SmartThings Hub V2 had three partitions in total, one unencrypted and two encrypted. The V2 hub used a securely stored encryption key in the unencrypted partition to encrypt and decrypt the second and third partitions of the eMMC flash storage. The SmartThings Hub V3 also had encrypted data at rest which was verified using `cryptsetup luksDump` tool.

For Two Espressif devices, the Orbit b-Hyve and the Wyze Lock Gateway, we reviewed the flash images using the `hexdump` and `strings` commands. Manual inspection of the flash image with these tools suggested the flash was not encrypted. We also used `Binwalk` to evaluate the entropy of the images. As there are few to no locations showing an entropy very close to one, we concluded with confidence that the flash images of these two devices were not encrypted. Other Linux based devices such as the Amcrest ProHD Shield and the Victure cameras utilized `squashfs` and or JFFS2 file systems. These are common file systems used with NAND flash chips. The file systems were extracted using tools such as `Binwalk` or `unsquashfs`. Similar to the analysis of the Amazon Echo smart speaker, if the device boot log was output over UART, it was inspected to determine information about the flash partition map.

C. Secure Boot

Table III shows the results from our analysis of secure boot. Results are grouped by whether or not the device had a chip capable of secure boot. Devices with a ✓ in the "Secure Boot Enabled SoC/MCU" column were verified to have support for secure boot from the device datasheet or the manufacturer's website. An × in the same column means otherwise. A ✓ in the "Secure Boot" column represents a device which we verified to be using secure boot. While an × meant we verified the device was not using secure boot, or the device did not properly implement secure boot.

TABLE IV: Summary of data at rest encryption analysis.

Device Category		Count	% Category	% Total
Crypto-enabled	Encrypted	2	20.0%	12.5%
	Unencrypted	7	70.0%	43.8%
	Inconclusive	1	10.0%	6.3%
	Total	10	100%	62.5%
Non crypto-enabled	Unencrypted	6	100%	37.5%
Vulnerable to data compromise		13	-	81.3%

Nine out of 16 devices had a SoC or MCU capable of secure boot; however, more than half of the 16 devices did not use or implement secure boot properly. For Amazon Echo Smart Speaker, we were able to boot from an SD card into our own U-Boot bootloader and then modify the device root file system, proving it did not properly implement secure boot. Looking at the Amcrest ProHD Shield from the UART port, we found that it verifies the checksum of the Linux kernel image before booting into the kernel. The checksum only verifies the image is error free and can be modified just as easily as the kernel image and provides no protection against firmware modification attacks.

For Samsung SmartThings Hubs, we used Device Tree Compiler tool to convert various `*.fit` files in the unencrypted partition to a human readable format. Within these files was information about the LUKS container with image hash values and signatures. The formatting adhered to the requirements in U-Boot documentation for verifying images [28] with verified boot proving secure boot. For Espressif devices, the Orbit b-Hyve and the Wyze Lock Gateway, we had to view the first part of the flash image. According to documentation, a 32-byte secure digest would be stored in the flash at `0x0` if secure boot was enabled. On both devices with ESP32 chips, the value at `0x0` in the flash was `0xFFFFFFFF FFFFFFFF`. Therefore, these devices did not have secure boot enabled.

D. Summary

Table IV shows a summary of our analysis of encryption results. Ten out of 16 devices, a total of 62.5%, used crypto enabled chips. Only two out of these 10 devices encrypted data at rest. If the devices did not have crypto-enabled hardware, they did not implemented encryption of data at rest. The majority of the devices (11) overall, 81.3% in total, did not implement storage encryption, leaving them vulnerable to sensitive data compromise.

Similarly, Table V shows more than half of the devices used chips capable of secure boot. However, we determined at least four of those devices did not use or did not properly implement the functionality. In short, 11 devices were found to be vulnerable to a firmware modification attack.

From a design choice perspective, many devices were vulnerable to the evil maid attack. A total of six out of 16 manufacturers of the devices analyzed did not select crypto-enabled hardware. Seven of the 16 devices analyzed did not contain hardware capable of secure boot. Even if manufactur-

TABLE V: Summary of secure boot analysis.

Device Category		Count	% Category	% Total
Secure boot enabled	Evidence of secure boot	2	22.2%	12.5%
	Implemented improperly	4	44.5%	25.0%
	Inconclusive	3	33.3%	18.8%
	Total	9	100%	56.3%
Non secure boot enabled	No secure boot	7	100%	43.7%
Vulnerable to firmware modification		11	-	68.8%

ers did select hardware supporting these features, they most often did not use the capability. Only one device manufacturer, two devices (Samsung SmartThings V2 and V3), implemented some form of data encryption and secure boot protecting the confidentiality and integrity of the device.

VI. Conclusion

In this paper, we present procedures for analyzing secure boot and encryption of data at rest that we used on 16 IoT devices. We found that 13 of these devices were vulnerable to sensitive data compromise due to lack of encryption, and 11 were vulnerable to firmware modification attacks. The hardware capabilities of the devices are often underutilized. Although the results produce insights into IoT device security with regard to the evil maid attack some limitations do exist. Our method assumes full disk or partition encryption, potentially leading to false positives. In addition, sufficiently compressed data might be misidentified as encrypted. Devices without crypto-enabled hardware could still perform cryptographic operations. Assumptions about hardware presence might result in false positives in secure boot analysis. Potential consequences of these assumptions should be considered with applying these methods in future work. Despite these limitations, in practice, our method's limitations are addressed simply by identifying critical data in plain text or booting modified firmware. These approaches are practical and effective, as demonstrated in Section V-A. This research lays groundwork for improved user data protection in IoT security as the IoT landscape continues to expand at a rapid pace.

References

[1] I. Analytics, "State of IoT 2021 number of connected iot devices growing 9% to 12.3 billion globally, cellular iot now surpassing 2 billion," 2021. [Online]. Available: https://iot-analytics.com/number-connected-iot-devices/

[2] "Global IoT connections data volume 2019 and 2025." [Online]. Available: https://www.statista.com/statistics/1017863/worldwide-iot-connected-devices-data-size/

[3] K. Zetter, *Countdown to Zero Day: Stuxnet and the Launch of the World's First Digital Weapon*. New York City: Crown, 2014.

[4] A. Greenberg, *Sandworm: A New Era of Cyberwar and the Hunt for the Kremlin's Most Dangerous Hackers*. New York: Anchor Books, 2019.

[5] M. Antonakakis, T. April, M. Bailey, M. Bernhard, E. Bursztein, J. Cochran, Z. Durumeric, J. A. Halderman, L. Invernizzi, M. Kallitsis *et al.*, "Understanding the mirai botnet," in *26th USENIX security symposium (USENIX Security 17)*, 2017, pp. 1093–1110.

[6] L. H. Newman, "What We Know About Friday's Massive East Coast Internet Outage," *Wired*, 2016, section: tags. [Online]. Available: https://www.wired.com/2016/10/internet-outage-ddos-dns-dyn/

[7] "Evil maid attack," https://en.wikipedia.org/wiki/Evil_maid_attack, 2020.

[8] H. Jiang, R. Chang, L. Ren, and W. Dong, "Implementing a arm-based secure boot scheme for the isolated execution environment," in *2017 13th International Conference on Computational Intelligence and Security (CIS)*. IEEE, 2017, pp. 336–340.

[9] V. B. Kumar, N. Gupta, A. Chattopadhyay, M. Kasper, C. Krauß, and R. Niederhagen, "Post-quantum secure boot," in *2020 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2020, pp. 1582–1585.

[10] Y. Gui, A. S. Siddiqui, and F. Saqib, "Hardware based root of trust for electronic control units," in *SoutheastCon 2018*. IEEE, 2018, pp. 1–7.

[11] V. Banerjee, S. Hounsinou, H. Olufowobi, M. Hasan, and G. Bloom, "Secure reboots for real-time cyber-physical systems," in *Proceedings of the 4th Workshop on CPS & IoT Security and Privacy (CPSIoTSec '22)*, 2022, p. 27–33.

[12] J. A. Halderman, S. D. Schoen, N. Heninger, W. Clarkson, W. Paul, J. A. Calandrino, A. J. Feldman, J. Appelbaum, and E. W. Felten, "Lest we remember: cold-boot attacks on encryption keys," *Communications of the ACM*, vol. 52, no. 5, pp. 91–98, 2009.

[13] M. Henson and S. Taylor, "Memory encryption: A survey of existing techniques," *ACM Computing Surveys (CSUR)*, vol. 46, no. 4, pp. 1–26, 2014.

[14] P. A. Peterson, "Cryptkeeper: Improving security with encrypted ram," in *2010 IEEE International Conference on Technologies for Homeland Security (HST)*. IEEE, 2010, pp. 120–126.

[15] M. D. Corner and B. D. Noble, "Zero-interaction authentication," in *Proceedings of the 8th annual international conference on Mobile computing and networking*, 2002, pp. 1–11.

[16] M. Huber, J. Horsch, J. Ali, and S. Wessel, "Freeze and crypt: Linux kernel support for main memory encryption," *Computers & Security*, vol. 86, pp. 420–436, 2019.

[17] J. Götzfried, N. Dörr, R. Palutke, and T. Müller, "Hypercrypt: Hypervisor-based encryption of kernel and user space," in *2016 11th International Conference on Availability, Reliability and Security (ARES)*. IEEE, 2016, pp. 79–87.

[18] C. Cao, L. Guan, N. Zhang, N. Gao, J. Lin, B. Luo, P. Liu, J. Xiang, and W. Lou, "Cryptme: Data leakage prevention for unmodified programs on arm devices," in *Research in Attacks, Intrusions, and Defenses: 21st International Symposium, RAID 2018, Heraklion, Crete, Greece, September 10-12, 2018, Proceedings 21*. Springer, 2018, pp. 380–400.

[19] I. Nadir, H. Mahmood, and G. Asadullah, "A taxonomy of iot firmware security and principal firmware analysis techniques," *International Journal of Critical Infrastructure Protection*, p. 100552, 2022.

[20] N. S. Mtetwa, P. Tarwireyi, A. M. Abu-Mahfouz, and M. O. Adigun, "Secure firmware updates in the internet of things: A survey," in *2019 International Multidisciplinary Information Technology and Engineering Conference (IMITEC)*. IEEE, 2019, pp. 1–7.

[21] C. Bradley and D. Barrera, "Towards characterizing iot software update practices," in *Foundations and Practice of Security: 15th International Symposium, FPS 2022, Ottawa, ON, Canada, December 12–14, 2022, Revised Selected Papers*. Berlin, Heidelberg: Springer-Verlag, 2023, p. 406–422.

[22] C. Gao, L. Luo, Y. Zhang, B. Pearson, and X. Fu, "Microcontroller based iot system firmware security: Case studies," in *2019 IEEE International Conference on Industrial Internet (ICII)*. IEEE, 2019, pp. 200–209.

[23] D. Giese and G. Noubir, "Amazon echo dot or the reverberating secrets of iot devices," in *Proceedings of the 14th ACM Conference on Security and Privacy in Wireless and Mobile Networks*, 2021, pp. 13–24.

[24] "OWASP Internet of Things Project - OWASP." [Online]. Available: https://wiki.owasp.org/index.php/OWASP_Internet_of_Things_Project#tab=IoT_Top_10

[25] "Boot Mode Selection - ESP32 - — esptool.py latest documentation." [Online]. Available: https://docs.espressif.com/projects/esptool/en/latest/esp32/advanced-topics/boot-mode-selection.html

[26] "SPIFFS Filesystem - ESP32 - — ESP-IDF Programming Guide latest documentation." [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/storage/spiffs.html

[27] "esptool.py," Nov. 2022, original-date: 2014-09-06T14:50:29Z. [Online]. Available: https://github.com/espressif/esptool

[28] "u-boot/u-boot," https://github.com/u-boot/u-boot/blob/0cbeed4f6648e0e4966475e3544280a69ecb59d3/doc/uImage.FIT/, November 2023.